Flip Health Backend System Design Document (HLD & LLD)

Table of Contents

1. Overview
2. System Architecture
3. RBAC Concept & Role Definitions
4. High-Level Design (HLD)
5. Low-Level Design (LLD)
6. Database Design
7. Sequelize Models & Associations
8. Authentication Flow
9. Authorization Flow (RBAC)
10. Middleware Design
11. Route Protection Strategy
12. Microservices Consideration
13. Best Practices & Security
14. Conclusion

---

1. Overview

This document outlines the backend architecture for Flip Health, a comprehensive healthcare platform offering:

- Doctor Consultations – Patients can book appointments with specific doctors; doctors manage their availability and view consultation requests.
- Pharmacy Orders – Customers purchase medicines; shop owners manage inventory and monitor stock levels.
- Network Management – Administration of healthcare facilities, clinics, and associated resources.

The system is built using Node.js, TypeScript, Express.js, MySQL, and Sequelize ORM. It implements Role-Based Access Control (RBAC) to ensure fine-grained permissions across all modules. The design is modular, scalable, and prepared for future microservices decomposition.

2. System Architecture

The architecture follows a layered approach with clear separation of concerns.

```text
Client (Mobile / Web)
       │
       ▼
┌──────────────┐
│  API Gateway │ (Optional – rate limiting, routing, aggregation)
└──────────────┘
       │
       ▼
┌──────────────────────────────────────┐
│         Express API Server           │
├──────────────────────────────────────┤
│ • Authentication Layer (JWT)         │
│ • RBAC Middleware                     │
│ • Business Services / Controllers     │
└──────────────────────────────────────┘
       │
       ▼
┌──────────────────────────────────────┐
│         Database (MySQL)              │
│  • Users, Roles, Permissions          │
│  • Appointments, Orders, Inventory    │

└──────────────────────────────────────┘
```

Key Components:

- API Gateway (optional) – Can be introduced for load balancing, caching, or routing to microservices.
- Express Server – Hosts all business logic, authentication, and authorization.
- Authentication Layer – Issues and validates JWT tokens.
- RBAC Middleware – Intercepts requests to verify permissions.
- Business Services – Encapsulate domain logic for Consultations, Pharmacy, Network.
- Database – MySQL with normalized tables and foreign key constraints.

3. RBAC Concept & Role Definitions

RBAC is implemented using the standard User → Role → Permission model.
A permission is defined as a combination of a Resource and an Action (e.g., `consultation:create`).

3.1 Roles

Role	Description
Super Admin	Full system control – can manage all users, roles, and data across modules.
Admin	Manages network entities (clinics, hospitals) and can view system reports.
Patient	Books consultations, views medical history, orders medicines.
Doctor	Views assigned consultations, updates availability, manages patient notes.
Customer	Browses pharmacy catalog, places orders, tracks order status.
Shop Owner	Manages pharmacy inventory (stock levels), processes orders.

A single user may hold multiple roles (e.g., a user can be both a Patient and a Customer).

3.2 Resources & Actions

Resource	Actions (examples)
`user`	create, read, update, delete
`role`	create, read, update, delete
`consultation`	create (book), read (view), update (reschedule/cancel) , doctor-specific actions
`appointment`	view (doctor), update status (confirm/complete)
`availability`	manage (doctor)
`pharmacy`	manage shop details
`inventory`	view (shop owner), update (add stock)
`order`	create (customer), read (own), update status (shop owner)
`network`	create, read, update, delete (facilities)

3.3 Permission Matrix (Illustrative)

Role	consultation:create	consultation:read (own)	inventory:update	order:create	network:manage
Super Admin	✓	✓	✓	✓	✓
Admin	–	–	–	–	✓
Patient	✓	✓	–	–	–
Doctor	–	✓ (assigned)	–	–	–
Customer	–	–	–	✓	–
Shop Owner	–	–	✓	✓ (own shop)	–

4. High-Level Design (HLD)

The system is composed of several logical modules, each exposing RESTful APIs.

4.1 Component Diagram

```
┌──────────────────┐     ┌──────────────────┐     ┌──────────────────┐
│   Auth Service   │     │   User Service   │     │   Role Service   │
│  (login, token)  │     │ (profile, CRUD)  │     │ (permissions)    │
└────────┬─────────┘     └────────┬─────────┘     └────────┬─────────┘
         │                        │                        │
         └────────────────────────┴────────────────────────┴
                                  │                        │
         ┌────────────────────────▼────────────────────────┐
         │ Consultation Service   │ │   Pharmacy Service   │
         │ • appointments         │ │ • orders             │
         │ • doctor availability  │ │ • inventory          │
         │ • medical records      │ │ • shop management    │
         └────────────────────────┬────────────────────────┘
                                  │                        │
         ┌────────────────────────▼────────────────────────▼────────────┐
         │                Network Service                               │
         │        (facilities, clinics, hospitals)                      │
         └──────────────────────────┬──────────────────────────────────┘
                                    │
                        ┌───────────▼──────────┐
                        │    Database          │
                        │     (MySQL)          │
                        └──────────────────────┘
```

- Auth Service – Handles user login, password verification, JWT issuance.
- User Service – Manages user profiles (CRUD) and role assignments.

- Role Service – Manages roles, resources, actions, and permission assignments.
- Consultation Service – Core appointment booking, doctor schedules, consultation notes.
- Pharmacy Service – Product catalog, orders, inventory tracking.
- Network Service – Management of clinics, hospitals, and other facilities.

All services share the same database but are logically separated. In a microservices architecture, they could be deployed independently with their own databases.

4.2 Data Flow Example – Book a Consultation

1. Patient logs in → receives JWT containing user ID and permissions.
2. Patient sends `POST /api/consultations` with doctor ID, date, time.
3. Request passes through Auth Middleware (validates JWT) and RBAC Middleware (checks `consultation:create`).
4. Consultation Service:
    - Verifies doctor availability.
    - Creates an appointment record with status `pending`.
    - Optionally sends notification to the doctor.
5. Returns appointment details to patient.

. Low-Level Design (LLD)

This section details the internal flows and algorithms for key features.

5.1 Doctor Availability Management

Actors: Doctor
Endpoint: `PUT /api/doctors/:doctorId/availability`
Permission: `availability:manage` (assigned to Doctor role)

Flow:

1. Doctor sends a list of time slots (e.g., `{ "date": "2025-04-10", "slots": ["09:00","10:00"] }`).
2. Service validates that the doctor owns the availability record (or is the same user).
3. Existing slots for that date are replaced/merged based on business rules.
4. Updated availability is stored in the `doctor_availabilities` table.

5. Response returns the updated list.

5.2 Pharmacy Order Placement

Actors: Customer
Endpoint: `POST /api/orders`
Permission: `order:create`

Flow:

1. Customer sends cart items (product IDs, quantities) and shipping address.
2. Service:
    - Checks inventory levels for each item.
    - Reserves stock (or rejects if insufficient).
    - Calculates total amount.
    - Creates an order record with status `pending`.
    - Creates order line items.
    - Triggers an asynchronous task to notify the shop owner.
3. Returns order ID and summary.

5.3 Inventory Update by Shop Owner

Actors: Shop Owner
Endpoint: `PATCH /api/inventory/:productId`
Permission: `inventory:update`

Flow:

1. Shop owner sends new stock quantity (absolute or delta).
2. Service verifies that the shop owner owns the pharmacy associated with the product.
3. Updates the `inventory` record.
4. If stock falls below a threshold, a low-stock alert is generated (optional).

5.4 Network Facility Management

Actors: Admin, Super Admin
Endpoint: `POST /api/network/facilities`
Permission: `network:create`

Flow:

1. Admin submits facility details (name, address, contact, type).
2. Service validates input, ensures unique constraints.
3. Creates a record in `facilities` table.
4. Returns the created facility.

---

6. Database Design

The database is normalized to avoid redundancy and maintain integrity. Below are the core tables, including those for RBAC and domain entities.

6.1 RBAC Tables

These tables follow the design from the provided PDF, with minor adjustments.

users

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(255)	NO		NULL	
email	varchar(255)	NO	UNI	NULL	
password	varchar(255)	NO		NULL	

phone	varchar(255)	YES		NULL
status	tinyint(1)	NO		1
is_verified	tinyint(1)	NO		0
is_deleted	tinyint(1)	NO		0
createdAt	datetime	NO		NULL
updatedAt	datetime	NO		NULL

roles

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(255)	NO		NULL	
code	varchar(100)	NO	UNI	NULL	
description	varchar(255)	YES		NULL	
createdAt	datetime	NO		NULL	
updatedAt	datetime	NO		NULL	

user_roles (junction)

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
user_id	int	NO	MUL	NULL	
role_id	int	NO	MUL	NULL	
createdAt	datetime	NO		NULL	
updatedAt	datetime	NO		NULL	

resources

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(255)	NO		NULL	
code	varchar(100)	NO	UNI	NULL	
description	varchar(255)	YES		NULL	
createdAt	datetime	NO		NULL	

updatedAt	datetime	NO		NULL	

actions

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(255)	NO		NULL	
code	varchar(100)	NO	UNI	NULL	
description	varchar(255)	YES		NULL	
createdAt	datetime	NO		NULL	
updatedAt	datetime	NO		NULL	

resource_actions (junction)

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
resource_id	int	NO	MUL	NULL	
action_id	int	NO	MUL	NULL	

createdAt	datetime	NO		NULL	
updatedAt	datetime	NO		NULL	

permission_roles (junction)

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
role_id	int	NO	MUL	NULL	
resource_action_id	int	NO	MUL	NULL	
createdAt	datetime	NO		NULL	
updatedAt	datetime	NO		NULL	

7. Sequelize Models & Associations

Using Sequelize, we define models with proper associations.

7.1 RBAC Models

- User `belongsToMany(Role, { through: UserRole })`

- Role `belongsToMany(User, { through: UserRole })`
- Role `belongsToMany(ResourceAction, { through: PermissionRole })`
- Resource `hasMany(ResourceAction)`
- Action `hasMany(ResourceAction)`
- ResourceAction `belongsTo(Resource)` and `belongsTo(Action)`

7.2 Domain Models

- User `hasOne(Doctor)`, `hasMany(Appointment)` (as patient), `hasMany(Order)` (as customer), `hasMany(Pharmacy)` (as owner)
- Doctor `belongsTo(User)`, `hasMany(DoctorAvailability)`, `hasMany(Appointment)`
- DoctorAvailability `belongsTo(Doctor)`, `hasOne(Appointment)` (optional)
- Appointment `belongsTo(User, { as: 'patient' })`, `belongsTo(Doctor)`, `belongsTo(DoctorAvailability)`
- Pharmacy `belongsTo(User, { as: 'owner' })`, `hasMany(Product)`, `hasMany(Order)`
- Product `belongsTo(Pharmacy)`, `hasOne(Inventory)`, `hasMany(OrderItem)`
- Inventory `belongsTo(Product)`
- Order `belongsTo(User, { as: 'customer' })`, `belongsTo(Pharmacy)`, `hasMany(OrderItem)`
- OrderItem `belongsTo(Order)`, `belongsTo(Product)`
- Facility `belongsTo(User, { as: 'creator' })`

---

8. Authentication Flow

1. User sends `POST /auth/login` with `email` and `password`.
2. Server fetches user from database, verifies password using bcrypt.
3. If valid, the system retrieves all roles assigned to the user and compiles a set of permissions (resource + action pairs).
4. A JWT token is generated containing:
    - `userId`
    - `roles` (array of role codes)
    - `policies` – a simplified structure like `{ "resource:action": true }` or an array of `{ resource, actions }` for easy checking.
5. Token is returned to client.

JWT Payload Example:

```json
{
  "userId": 123,
    "roles": ["patient", "customer"],
"policies": {
    "consultation:create": true,
    "consultation:read": true,
    "order:create": true,
    "order:read": true
  },
  "iat": 1516239022,
  "exp": 1516242622
}
```

---

9. Authorization Flow (RBAC)

1. Client includes JWT in `Authorization: Bearer <token>` header.
2. For each protected endpoint, the Auth Middleware decodes and validates the token, attaching user info to the request object.
3. The RBAC Middleware (or a dedicated function) checks if the required permission (resource + action) exists in `req.user.policies`.
4. If present, request proceeds to the controller; otherwise, a `403 Forbidden` response is returned.

Example Check:

```typescript
// In route definition
router.post('/consultations',
  authMiddleware,
  rbacMiddleware('consultation', 'create'),
  consultationController.create
);
```

---

10. Middleware Design

10.1 `authMiddleware`

- Extracts token from `Authorization` header.
- Verifies token using secret key.
- If valid, decodes payload and attaches `req.user`.
- If invalid or missing, responds with `401 Unauthorized`.

10.2 `rbacMiddleware(resource, action)`

- Factory function that returns a middleware.
- Checks `req.user.policies` for the required permission.
- If not found, returns `403 Forbidden`.
- Otherwise calls `next()`.

10.3 Optional: `resourceOwnershipMiddleware`

- For operations that require ownership (e.g., patient viewing own appointments), this middleware can verify that the requested resource ID belongs to the authenticated user.
- Example: `appointmentBelongsToUser` checks `req.params.id` against `req.user.userId`.

---

11. Route Protection Strategy

Routes are protected by combining authentication and RBAC middlewares. Below are example route definitions.

Method	Endpoint	Required Permission	Description

POST	`/api/auth/login`	public	User login
GET	`/api/users/me`	(authenticated)	Get own profile
POST	`/api/consultations`	`consultation:create`	Book a consultation
GET	`/api/consultations`	`consultation:read`	List consultations (filtered by role)
PUT	`/api/doctors/:id/availability`	`availability:manage`	Set doctor availability
POST	`/api/orders`	`order:create`	Place an order
GET	`/api/orders`	`order:read`	List orders (own for customer, all for shop owner)
PATCH	`/api/inventory/:productId`	`inventory:update`	Update stock
POST	`/api/network/facilities`	`network:create`	Add a new facility
DELETE	`/api/network/facilities/:id`	`network:delete`	Remove a facility

Note: Additional checks (e.g., ownership) may be applied inside controllers.

---

12. Microservices Consideration

The design is modular enough to be split into independent microservices. Each domain (Auth, User, Consultation, Pharmacy, Network) could become a separate service with its own database. Communication between services would then occur via REST APIs or message queues.

Potential Microservices Boundaries:

- Auth Service – Handles login, token issuance, and user credential management.
- User Service – Manages user profiles and role assignments.
- Consultation Service – Manages doctors, availability, appointments.
- Pharmacy Service – Manages pharmacies, products, inventory, orders.
- Network Service – Manages facilities.

Challenges & Solutions:

- Data Consistency – Use distributed transactions (Saga pattern) or eventual consistency.
- Service Discovery – Implement a service registry (e.g., Consul, Eureka).
- API Gateway – Route requests to appropriate services, handle authentication centrally.

---

13. Best Practices & Security

- Password Hashing – Use bcrypt with salt rounds (≥10).
- JWT Security – Sign tokens with a strong secret; set short expiration; use HTTPS.
- Input Validation – Validate all inputs using libraries like Joi or class-validator.
- SQL Injection Prevention – Use Sequelize ORM (parameterized queries).
- Rate Limiting – Implement rate limiting (e.g., express-rate-limit) to prevent abuse.
- Audit Logging – Log critical actions (user creation, permission changes) for compliance.
- Environment Variables – Store secrets in `.env` files, never in code.
- Error Handling – Return generic error messages to client; log detailed errors internally.
- CORS – Configure appropriately for frontend domains.
- Data Encryption – Encrypt sensitive data at rest if required.

---

14. Conclusion

This document provides a comprehensive high-level and low-level design for the Flip Health backend system. The architecture leverages Node.js, Express, MySQL, and Sequelize with a robust RBAC model to support doctor consultations, pharmacy operations, and network management. The design is scalable, secure, and ready for implementation, with clear pathways for future evolution into a microservices architecture. All key user stories (patient booking, doctor availability, pharmacy ordering, inventory management, and super admin controls) are supported through well-defined APIs and database structures.